

GRAPH

DATA STRUCTURES

Circular Queue

Linked List

Doubly Linked List

Binary Heap

Binary Search Tree

AVL Tree

Red-Black Tree

AVL Tree vs Red-Black Tree

Splay Tree

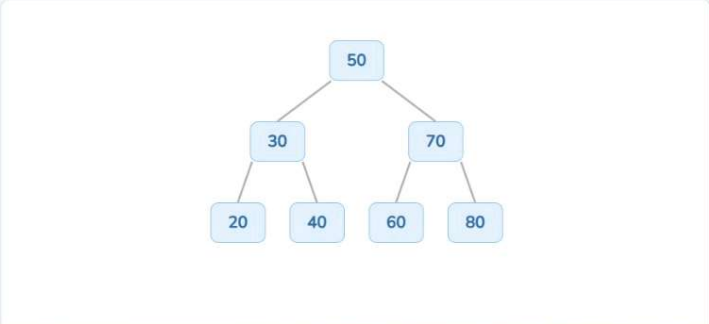
B-Tree

B-Tree Visualization

A **B-Tree** is a self-balancing search tree designed to maintain sorted data and allow efficient insertion, deletion, and search operations. Unlike [binary trees](#), each node can hold multiple keys and have more than two children. B-Trees are widely used in databases and file systems where large blocks of data must be read and written efficiently.

When inserting a new key, it is placed into the appropriate leaf node. If the node overflows by exceeding the maximum number of keys, it **splits** — the median key is pushed up to the parent, while the remaining keys form two child nodes. This process can propagate upward and may even create a new root. This splitting mechanism is essential for keeping the tree balanced, ensuring that all leaves always remain at the same depth as keys are redistributed.

Enter a number: 21 ▶ INSERT SEARCH CLEAR ↶ ↷ ↺ 📄 🔗



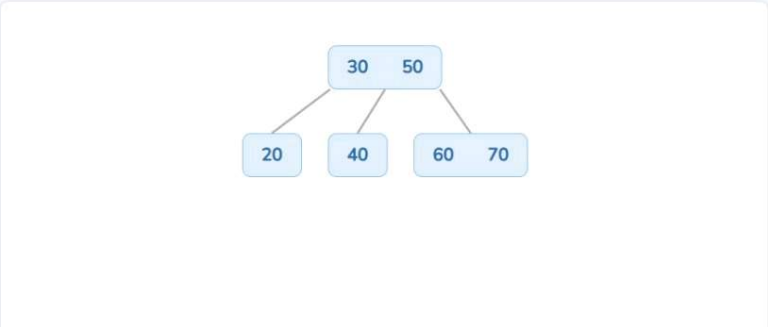
- GRAPH
- DATA STRUCTURES
- Circular Queue
- Linked List
- Doubly Linked List
- Binary Heap
- Binary Search Tree
- AVL Tree
- Red-Black Tree
- AVL Tree vs Red-Black Tree
- Splay Tree
- B-Tree

B-Tree Visualization

A **B-Tree** is a self-balancing search tree designed to maintain sorted data and allow efficient insertion, deletion, and search operations. Unlike [binary trees](#), a node can hold multiple keys and have more than two children. B-Trees are widely used in databases and file systems where large blocks of data must be written efficiently.

When inserting a new key, it is placed into the appropriate leaf node. If the node overflows by exceeding the maximum number of keys, it **splits** — the middle key is pushed up to the parent, while the remaining keys form two child nodes. This process can propagate upward and may even create a new root. This mechanism is essential for keeping the tree balanced, ensuring that all leaves always remain at the same depth as keys are redistributed.

Enter a number:



- SEE ALGORITHMS
- GRAPH
- DATA STRUCTURES
- Circular Queue
- Linked List
- Doubly Linked List
- Binary Heap
- Binary Search Tree
- AVL Tree
- Red-Black Tree
- AVL Tree vs Red-Black Tree
- Splay Tree
- B-Tree

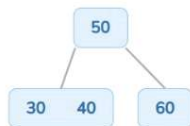
B-Tree Visualization

A **B-Tree** is a self-balancing search tree designed to maintain sorted data and allow efficient insertion, deletion, and search operations. Unlike [binary trees](#), each node can hold multiple keys and have more than two children. B-Trees are widely used in databases and file systems where large blocks of data must be read and written efficiently.

When inserting a new key, it is placed into the appropriate leaf node. If the node overflows by exceeding the maximum number of keys, it **splits** — the median key is pushed up to the parent, while the remaining keys form two child nodes. This process can propagate upward and may even create a new root. This splitting mechanism is essential for keeping the tree balanced, ensuring that all leaves always remain at the same depth as keys are redistributed.

Enter a number: 97

▶ INSERT SEARCH CLEAR ↶ ↷ ↺ ⏏ ↻



GRAPH

DATA STRUCTURES

Circular Queue

Linked List

Doubly Linked List

Binary Heap

Binary Search Tree

AVL Tree

Red-Black Tree

AVL Tree vs Red-Black
Tree

Splay Tree

B-Tree

B-Tree Visualization

A **B-Tree** is a self-balancing search tree designed to maintain sorted data and allow efficient insertion, deletion, and search operations. Unlike [binary trees](#), each node can hold multiple keys and have more than two children. B-Trees are widely used in databases and file systems where large blocks of data must be read and written efficiently.

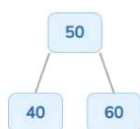
When inserting a new key, it is placed into the appropriate leaf node. If the node overflows by exceeding the maximum number of keys, it **splits** — the median key is pushed up to the parent, while the remaining keys form two child nodes. This process can propagate upward and may even create a new root. This splitting mechanism is essential for keeping the tree **balanced**, ensuring that all leaves always remain at the same depth as keys are redistributed.

Enter a number: 

INSERT

SEARCH

CLEAR



- DATA STRUCTURES
- Linear Queue
- Linked List
- Doubly Linked List
- Binary Heap
- Binary Search Tree
- B+ Tree
- B-Tree
- Black Tree
- B+ Tree vs Red-Black Tree
- AVL Tree

B-Tree Visualization

A **B-Tree** is a self-balancing search tree designed to maintain sorted data and allow efficient insertion, deletion, and search operations. Unlike [binary trees](#), each node can hold multiple keys and have more than two children. B-Trees are widely used in databases and file systems where large blocks of data must be read and written efficiently.

When inserting a new key, it is placed into the appropriate leaf node. If the node overflows by exceeding the maximum number of keys, it **splits** — the median key is pushed up to the parent, while the remaining keys form two child nodes. This process can propagate upward and may even create a new root. This splitting mechanism is essential for keeping the tree balanced, ensuring that all leaves always remain at the same depth as keys are redistributed.

Enter a number: ▶ INSERT SEARCH CLEAR ↶ ↷ ↺ ↻ 📄 🔗

